

SkyGuard AI — Judge pitch

Team Think Tank · Problem Statement SIH26073

AI/ML-Based Intelligent Anomaly Detection for Automatic Weather Stations

Clean · teal

Genuine weather · amber

Hardware · rose

Unconfirmed · slate

Before you walk in. API on : 8000, streamer on Palam + buddies, dashboard on : 8501. Land on **Network**. Confirm header chips **ok · model_loaded · n_stations = 151**. Roster shows Palam (42181), Safdarjung (42182), Meerut (42139), Santacruz (43003) — all teal. “Include 1-hop buddies in ingest” stays on. Do not open Advanced inject unless a judge asks.

● Spoken lines — say these ● Green bars — click / point

0:00 – 0:25 Team intro

Network already open · do not click

Good morning, respected judges. We are Team Think Tank — and we're here for Problem Statement SIH26073: AI/ML-Based Intelligent Anomaly Detection for Automatic Weather Stations, under the Disaster Management theme.

Every IMD forecast, flood model, and heat advisory in this country starts with an Automatic Weather Station. If that hour is wrong, the model is wrong — and nobody on the ground knows until the damage is already done.

0:25 – 1:05 The problem

Stay on Network · gesture at the map

Point at the India map and the KPI strip: Clean / Weather / Hardware / Unconfirmed / Waiting. Do not click yet.

This is **SkyGuard AI** — live quality control for Indian AWS. We are allowed **only three channels**: temperature, pressure, humidity. No radar. No satellite. No wind.

A min/max gate cannot do this job. A frozen sensor at 28.4 °C looks like “stable weather.” A real monsoon cell looks like a broken instrument. The official SIH example is Palam at 55 °C while the rest of NCR is normal — that is hardware, not a heatwave. The product has to tell those two stories apart **in the same hour**.

What you're looking at is not four toy stations. The LSTM was trained on **151 Indian stations**. The live view is a handful so the map stays readable. Palam, Safdarjung, and Meerut sit together in Delhi-NCR. Santacruz is Mumbai — and **Delhi does not validate Mumbai**.

Header chips: the model is loaded. Color is **this hour's QC label**, not health. Teal means we trust it.

1:05 – 1:40 How it thinks

Stay on Network · point at the color legend

Three tiers. One engine. The dashboard never computes — it only polls.

One. Physical rules: impossible range, a 10 °C one-hour jump, a missing packet. Hard fail. We don't need neighbors for that.

Two. An LSTM autoencoder on the last **24 hours**, with a **per-station** scaler. Palam's “normal day” is not Mumbai's. Unusual reconstruction error means *candidate* — including real weather.

Three. A buddy check on the trained graph. Inverse-distance weights. **At least two** usable neighbors at the same hour. If they agree → **genuine weather, amber, health unchanged**. If they disagree → **hardware, rose**. If we don't have two neighbors, we say **unconfirmed**. We refuse to guess.

The streamer only tells the truth. Live faults are armed here, before the model sees the hour. Raw T, P, H are **never overwritten**.

Sidebar → Demo → Control. Caption: “The streamer stays clean.” Click Storm around Palam. Wait 1–2 seconds.

This page is **Control**. These three buttons are the judge controls.

If Palam sits grey for a second, ignore it. Palam just arrived this hour — the buddy check waits until Safdarjung and Meerut are in too, then it should go amber.

Sidebar → Network. Point: Palam + Safdarjung + Meerut amber. Santacruz still teal. KPI Weather has moved. Roster health has not crashed.

Palam, Safdarjung, and Meerut all dropped together — temperature down, pressure down, humidity up. Neighbors agree. That is a **genuine weather event**. Amber — **never red**. A cyclone must not send Palam to the workshop.

Santacruz is still teal. We did not paint all of India. The storm is Palam’s **neighborhood**, not a metro cluster named NORTH.

2:35 – 3:20 **Story 2 — a broken sensor**

Control → Network

Control → Reset overlays. Wait until markers would be teal. Then Break Palam temperature. Back to Network.

Same network. One station. One channel — temperature. Neighbors did **not** move. That is a **hardware anomaly**. Only Palam goes rose.

That is the whole product, in two clicks: **a storm around Palam is not a broken sensor**.

3:20 – 4:10 **Proof we didn’t cheat**

Station → Alerts

Click Palam on the roster (or the Palam marker). Station opens.

This is **Station**. Solid line is **observed** — the spike is still there. We do not “correct” the forensic record. Dashed line is the LSTM reconstruction for *this* hour, not a forecast of tomorrow.

The verdict sentence is the reason the engine wrote — temperature observed versus predicted, neighbors disagree, treated as hardware. Contribution bars name **temp_c**. Health here is a **7-day maintenance index**. Weather does not lower it. A spike does.

Sidebar → Alerts. Newest first. Optional: click Open on a rose row to pin that hour on Station.

Newest first. Amber weather versus rose hardware. Same sentence you just saw. **Open** pins that hour on Station so an operator can inspect it later — even after the live hour has gone clean.

We also catch freeze, drift, and comms drop from Advanced inject. Same engine. We are not showing those because the judge question is weather versus hardware.

Open **How QC works** only if a judge looks lost. Do not wander into Advanced.

Why this matters under Disaster Management: IMD and state agencies cannot wait for a human to notice a silent AWS. A false cyclone alert wastes response. A missed sensor fault poisons the next flood model. SkyGuard sits in front of the network and answers, every hour: **trust this, or don't — and why.**

We built the full loop: 151-station catalog, clean historical replay, a FastAPI shell that never overwrites raw, production QC in-process — rules, LSTM, buddy graph — and this operator console. Isolates stay unconfirmed on purpose. We would rather say “not enough evidence” than invent a spatial call.

SkyGuard is not a weather forecast. It is a **self-aware, self-healing weather network**: detect, overlay, keep the raw, track health, keep the trusted stations in the operational mesh.

Thank you. We're happy to take questions — or inject a freeze or a comms drop live if you'd like to see the other faults.

CLICK SHEET — TAPE THIS TO THE LAPTOP

TIME	PAGE	DO THIS
Start	Network	Nothing. Confirm teal roster + header chips.
~1:40	Control	Click Storm around Palam.
~1:50	Network	Point amber NCR cluster; Santacruz stays teal.
~2:40	Control	Reset overlays → wait teal → Break Palam temperature.
~2:55	Network	Only Palam is rose. Safdarjung stays teal.
~3:20	Station	Open Palam from the roster. Solid observed / dashed overlay / verdict / contribution.
~3:50	Alerts	Amber vs rose. Optional Open.
Close	Stay put	Do not open Advanced inject unless asked.

IF SOMETHING GLITCHES

Slate on the first storm hour

Say: “Palam just arrived this hour. We wait for Safdarjung and Meerut, then it should go amber.” Skip the line if all three go amber immediately.

Everything unconfirmed

Seed or 24h window missing. Don't debug on stage. Open **How QC works** and walk the three tiers.

Santacruz never painted

That's the win. Say it: Delhi does not validate Mumbai.

Words we do not say

NORTH / WEST. “We overwrite the reading.” “The dashboard detects anomalies.” Weather is never red.

Spoken length is about 4:20 if you don't rush, ~5:00 with the Station pause. Rehearse once with the 200 ms stream so you know how long amber and rose take to land.